

时间序列预测项目说明

1 项目概述

本项目实现了多种时间序列预测模型,包括传统基线模型和基于 TCLN(Transformer-based Conv-LSTM Network) 的变体。项目支持严格按论文实现的 TCLN 版本, 以及带有 StarNet/Hadamard 融合的变体。

2 环境准备

2.1 1. 安装依赖

```
pip install -r requirements.txt
```

主要依赖包括:

- torch - PyTorch 深度学习框架
- numpy - 数值计算
- pandas - 数据处理和结果保存
- scikit-learn - 数据预处理
- fvcare / thop - 模型 FLOPs 分析 (可选)

2.2 2. 数据准备

项目支持多个标准数据集, 默认路径结构:

```
all_six_datasets/  
  ETT-small/  
    ETTh2.csv  
  electricity/  
    electricity.csv  
  exchange_rate/  
    exchange_rate.csv  
  illness/  
    national_illness.csv  
  traffic/  
    traffic.csv  
  weather/  
    weather.csv
```

数据会自动下载或放置在上述路径中, 代码会自动识别。

3 快速开始

3.1 1. 运行单个模型

```
# 运行 LSTM 模型
python -m tsproj.run_experiments --dataset ETTh2 --horizon 8 --model lstm

# 运行带 StarNet 融合的 LSTM
python -m tsproj.run_experiments --dataset ETTh2 --horizon 8 --model lstm_star

# 运行严格版 TCLN
python -m tsproj.run_experiments --dataset ETTh2 --horizon 8 --model tcln_strict

# 运行严格版 TCLN + StarNet
python -m tsproj.run_experiments --dataset ETTh2 --horizon 8 --model
    tcln_strict_star
```

3.2 2. 批量运行所有模型

```
# 运行所有模型在 ETTh2 数据集上
python -m tsproj.run_all --dataset ETTh2

# 跳过已存在的结果(继续运行)
python -m tsproj.run_all --dataset ETTh2 --skip-existing
```

3.3 3. 运行特定模型组合

```
# 只运行指定的模型
python -m tsproj.run_all --dataset ETTh2 --models "lstm,lstm_star,tcln_strict,
    tcln_strict_star"

# 指定预测步长
python -m tsproj.run_all --dataset ETTh2 --horizons "8,16,32"
```

4 支持的模型

4.1 传统基线模型

- lstm / lstm_{star} – LSTM StarNet gru/gru_{star} – GRU StarNet
- transformer / transformer_{star} – Transformer StarNet cnn1d/cnn1d_{star} – CNN StarNet
- da_{rnn}/da_{rnn}_{star} – DA – RNN StarNet tpa_{lstm}/tpa_{lstm}_{star} – TPA – LSTM StarNet
- da_{convlstm}/da_{convlstm}_{star} – DA – Conv – LSTM StarNet ts_{convlstm}/ts_{convlstm}_{star} – TS – Conv – LSTM StarNet

4.2 TCLN 变体

- $tcln_{plain} = TCLN(bypass)tcln_{starnet} - TCLN + StarNet$
- $tcln_{strict} = TCLN()tcln_{strictstar} - TCLN + StarNet$

4.3 研究模型

- $img_{cnnlstm} = CNN - LSTM_{time2d_{starnet}} - 2D - StarNet$

5 模型参数说明

5.1 通用参数

- `-dataset` - 数据集名称 (ETTh2, electricity, exchange_rate, illness, traffic, weather) -- `-horizon-`
- `-model` - 模型名称

5.2 图像序列模型专用参数

- `-forced-p` - 强制指定图像尺寸 p (默认自动计算)
- `-feature-stride` - 特征采样步长 (默认 1)
- `-time-stride` - 时间采样步长 (默认 1)

5.3 时间 2D 模型专用参数

- `-window-points` - 时间窗口点数 (默认 25)
- `-upsample-to` - 上采样尺寸 (可选)

6 输出结果

6.1 1. 训练日志

每个模型运行会生成 CSV 文件，路径结构如下：

```
results/  
  lstm_ETTh2_h8.csv  
  lstm_star_ETTh2_h8.csv
```

CSV 包含每轮的训练损失、验证指标、学习率、运行时间等。

6.2 2. 汇总结果

批量运行会生成汇总文件，路径结构如下：

```
results/  
  summary_all_ETTh2.csv  
  summary.csv
```

汇总包含所有模型的测试指标、参数量、FLOPs、推理时间等。

7 配置说明

主要配置在 `tsproj/config.py` 中:

```
@dataclass
class ExperimentConfig:
    seq_len: int = 96          # 输入序列长度
    batch_size: int = 32       # 批大小
    epochs: int = 50           # 训练轮数
    base_lr: float = 1e-3      # 基础学习率
    weight_decay: float = 1e-4 # 权重衰减
    # ... 其他配置
```

8 严格 TCLN 说明

`tclnstrict`

Multi-kernel CNN: 每个时间步独立处理

Encoder 输出: 收集每层的 $O_1, O_2, \dots, O_n$

AR 组件: 使用目标变量的历史值 y_{hist} Decoder : $ReLU(concat(O_1..O_n, O_A R)) \rightarrow Linear$
推荐使用 `tclnstrict tclnstrictstar`

9 故障排除

9.1 1. CUDA 相关问题

```
# 检查 CUDA 可用性
python -c "import torch; print(torch.cuda.is_available())"

# 强制使用 CPU
export CUDA_VISIBLE_DEVICES=""
```

9.2 2. 数据路径问题

确保数据文件在正确路径, 或修改 `tsproj/data.py` 中的 `defaultdatasetpaths()`

9.3 3. 内存不足

减少 `batchsize` `--forced-p`

10 项目结构

```
tsproj/
  __init__.py
  config.py      # 实验配置
  data.py        # 数据加载和预处理
  train.py       # 训练和评估逻辑
  run_experiments.py # 单个实验运行
  run_all.py     # 批量实验运行
```

```
profile.py          # 模型分析工具
starnet.py          # StarNet 组件
models/
  __init__.py
  baselines.py      # 基线模型
  starnet.py        # StarNet 组件变体
  tcln.py
  tcln_strict.py    # 严格 TCLN 实现
```